
Digital Signal Processing

Filter Structure/Implementation

Implementation

PART II

Structure for discrete-time

- “filter”: DT system
- Structures ?
- block diagrams/ signal flow graphs
- Many possible structures for one filter
 - Direct forms
 - Cascade form
 - Parallel form
 - Transposed forms

Realization of Filters

- FIR or IIR filters with *Rational* TF: difference equation

$$y[n] = \sum_{k=0}^M b_k x[n-k] + \sum_{k=0}^N a_k y[n-k]$$

- Implementation: software or hardware
 - limited precision effects in digital implementations (round-off noise, overflow, . . .)
 - stability of desired properties (e.g. linear-phase)
 - minimization of the required resources

Digital Filter

- General N^{th} -order difference equation:

$$y[n] = \sum_{k=1}^N a_k y[n-k] + \sum_{k=0}^M b_k x[n-k]$$

- System function:

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 - \sum_{k=1}^N a_k z^{-k}} = A \frac{\prod_{k=1}^M (1 - c_k z^{-1})}{\prod_{k=1}^N (1 - d_k z^{-1})}$$

Direct Form I Implementation

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 - \sum_{k=1}^N a_k z^{-k}} = \underbrace{\left(\frac{1}{1 - \sum_{k=1}^N a_k z^{-k}} \right)}_{\text{poles}} \underbrace{\left(\sum_{k=0}^M b_k z^{-k} \right)}_{\text{zeros}}$$

$$Y(z) = H(z)X(z)$$

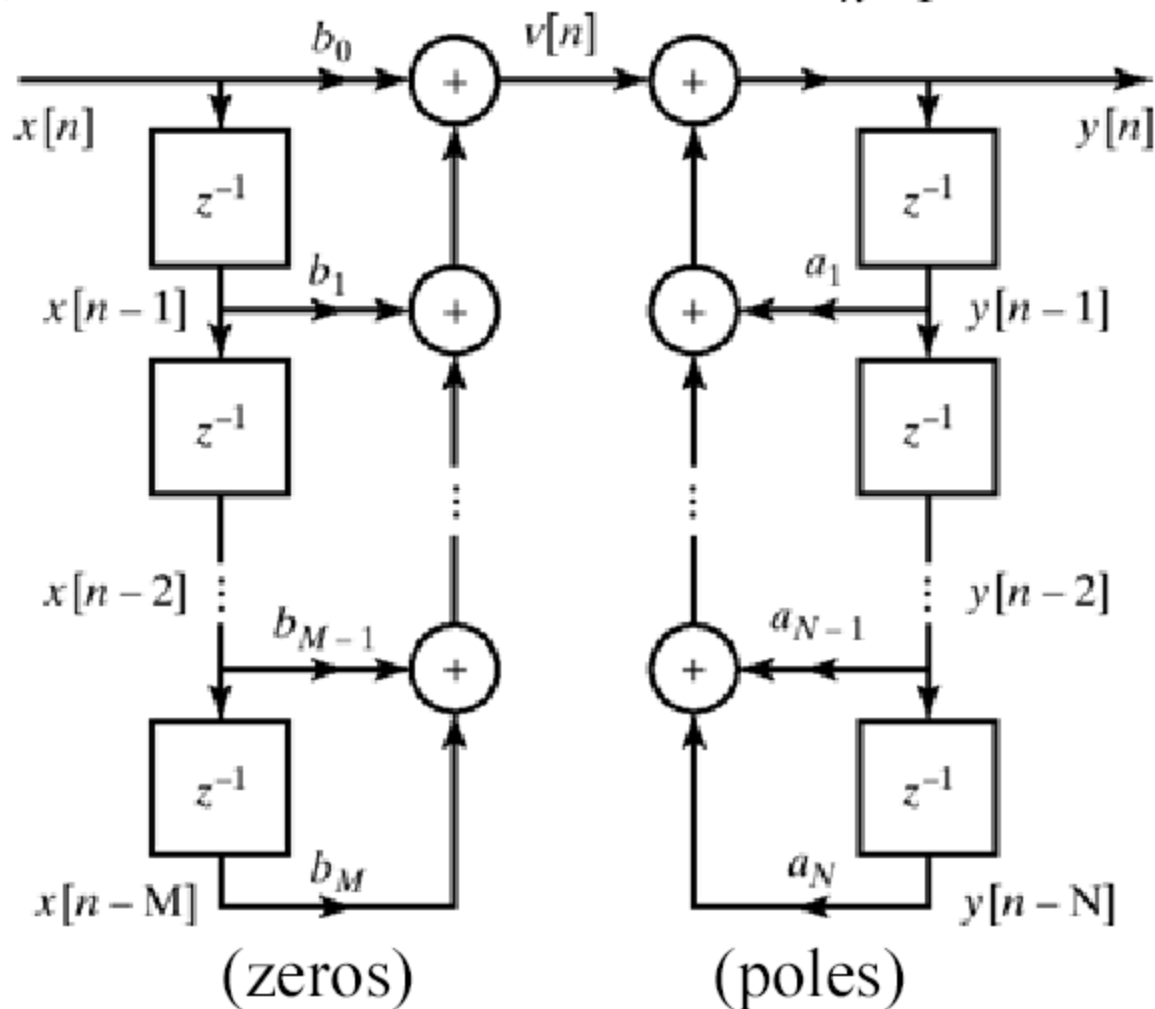
$$v[n] = \sum_{k=0}^M b_k x[n-k] \quad (\text{zeros})$$

$$y[n] = \sum_{k=1}^N a_k y[n-k] + v[n] \quad (\text{poles})$$

Block Diagram IIR DF-I

$$v[n] = \sum_{k=1}^M b_k x[n-k]$$

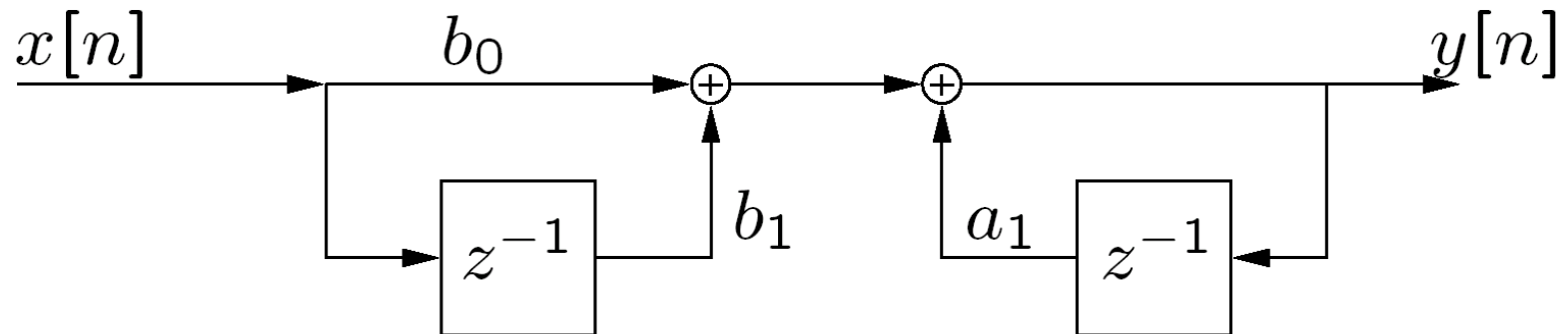
$$y[n] = \sum_{k=1}^N a_k y[n-k] + v[n]$$



Realization of Filters: Ex.

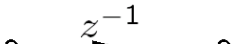
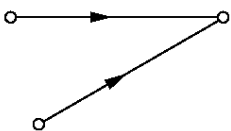
$$y[n] = b_0x[n] + b_1x[n-1] + a_1y[n-1]$$

$$H(z) = \frac{b_0 + b_1z^{-1}}{1 - a_1z^{-1}}$$



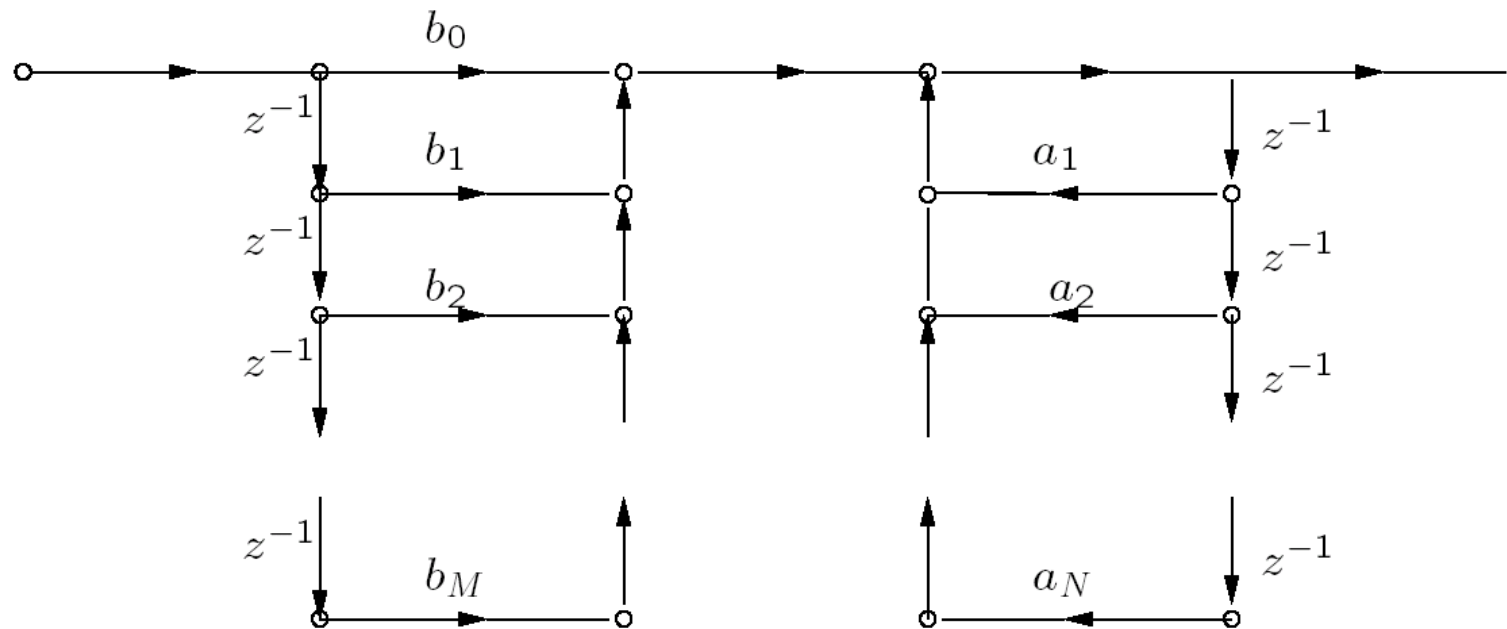
Block Diagrams / signal Flowgraphs

Basic elements:

	Block diagram	Flowgraph
scalar multiplication	$x[n] \xrightarrow{a} ax[n]$	
delay	$x[n] \xrightarrow{z^{-1}} x[n-1]$	
addition	$x_1[n] \xrightarrow{\quad} \oplus \xrightarrow{\quad} y[n]$ $x_2[n] \uparrow$	

Signal Flow Graph: DF-I

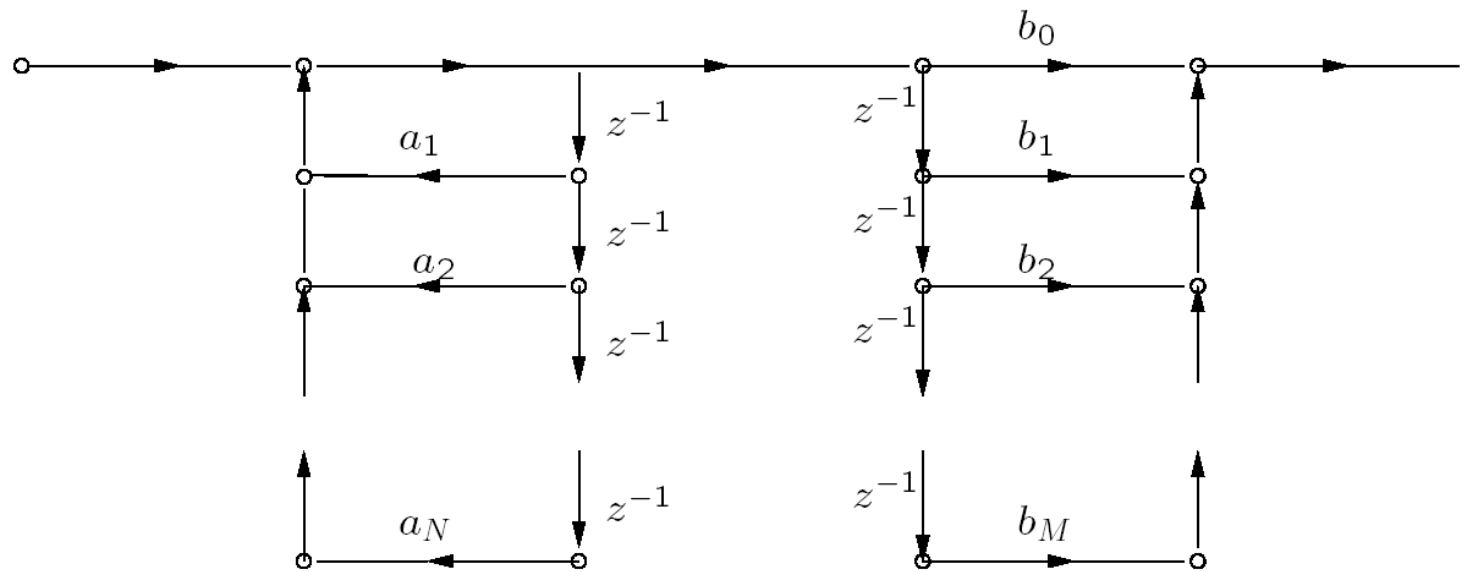
$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 - \sum_{k=1}^N a_k z^{-k}}$$



Multiple Structures

- all equivalent analytically
- all different w.r.t.
 - resources (number of delays, adders, multipliers)
 - robustness
 - error propagation, overflow

Discrete Form II

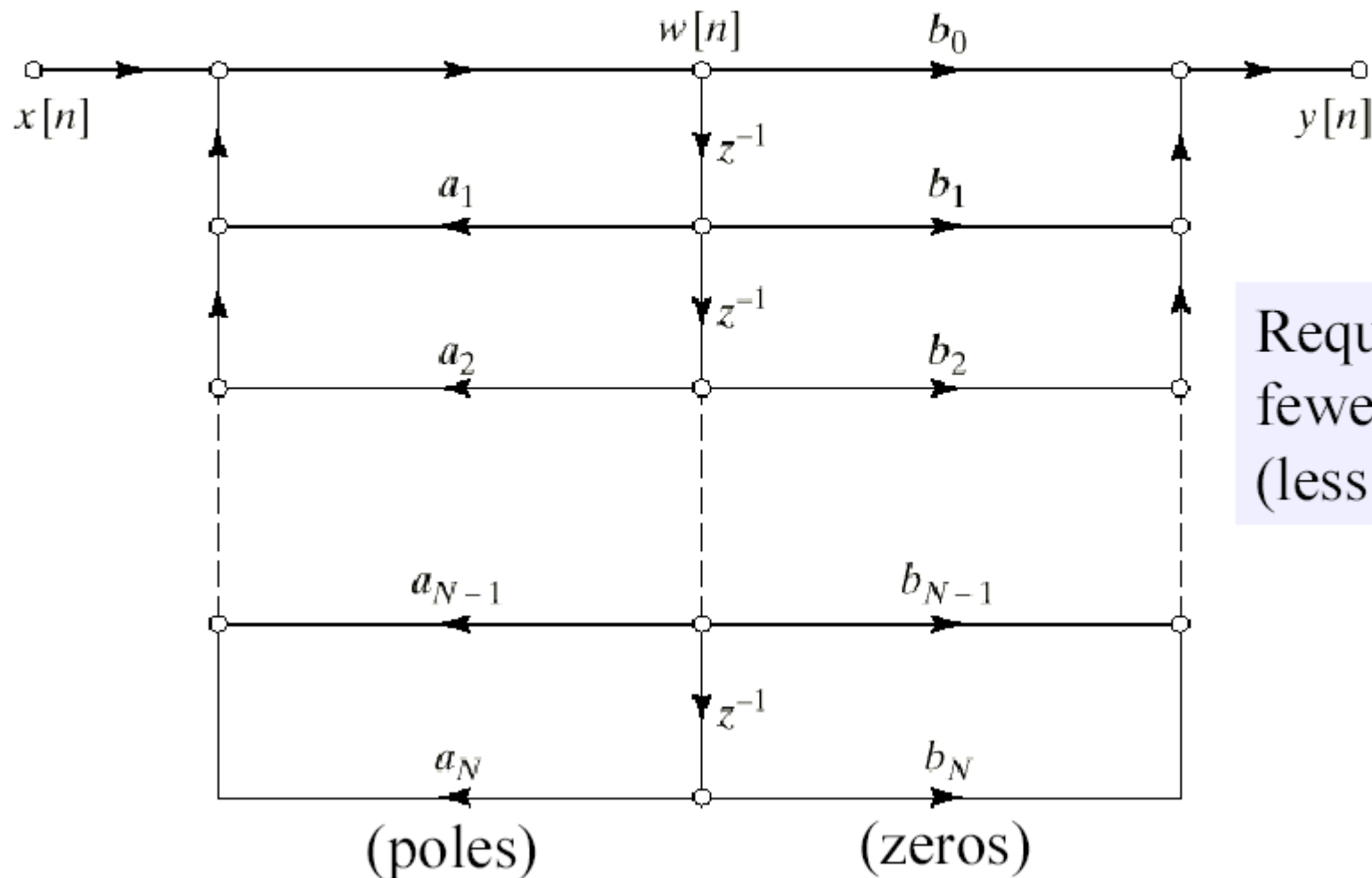


Canonic Form : minimal number of delays

Signal Flow Graph: IIR DF-II

$$w[n] = \sum_{k=1}^N a_k w[n-k] + x[n]$$

$$y[n] = \sum_{k=1}^M b_k w[n-k]$$



Requires
fewer delays
(less memory)

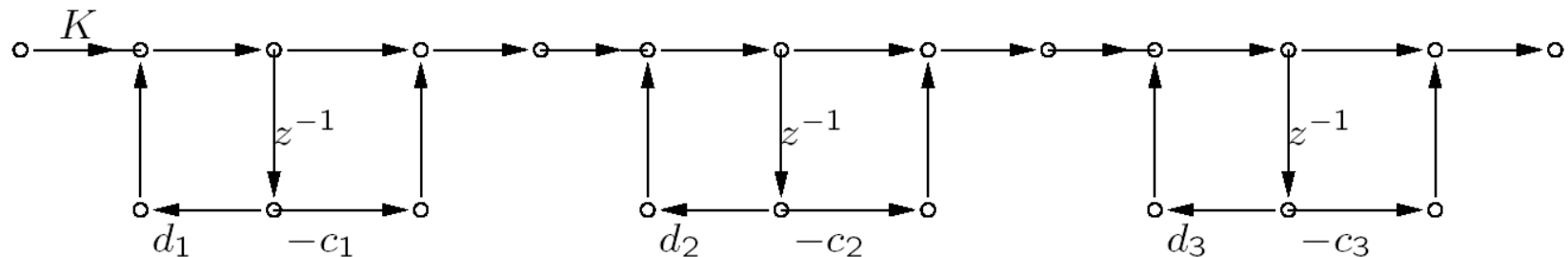
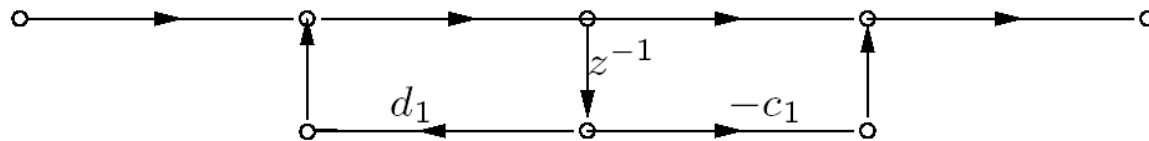
Discrete Form II (canonic)

Properties of “canonical” forms:

- N delay elements.
- $2N$ (2-input) adders.
- $2N + 1$ multipliers.

Cascade Form

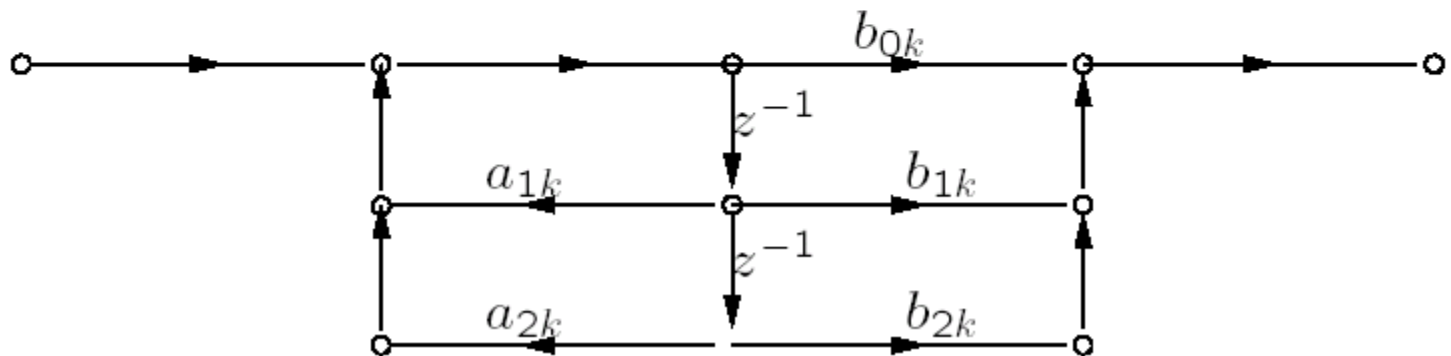
$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 - \sum_{k=1}^N a_k z^{-k}} = K \frac{\prod_{k=1}^M 1 - c_k z^{-1}}{\prod_{k=1}^N 1 - d_k z^{-1}}$$



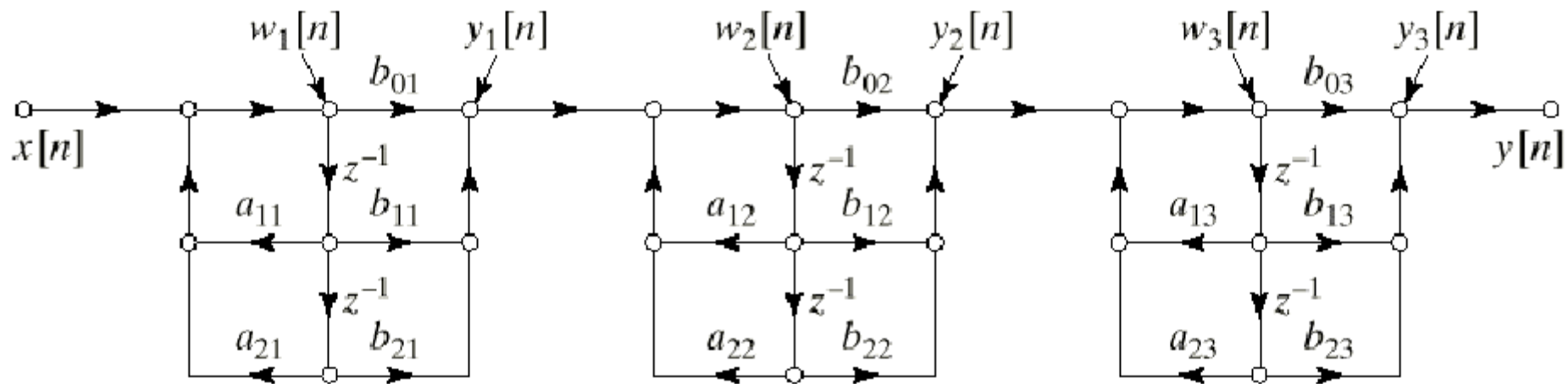
Cascade Form: Real Case

$$H(z) = \prod_{k=1}^{N_s} \frac{b_{0k} + b_{1k}z^{-1} + b_{2k}z^{-2}}{1 - a_{1k}z^{-1} - a_{2k}z^{-2}}$$

Second-order section:



IIR Cascade Form



$$H(z) = \prod_{k=1}^{N_s} \frac{b_{0k} + b_{1k}z^{-1} + b_{2k}z^{-2}}{1 - a_{1k}z^{-1} + a_{2k}z^{-2}}$$

$$y_0[n] = x[n],$$

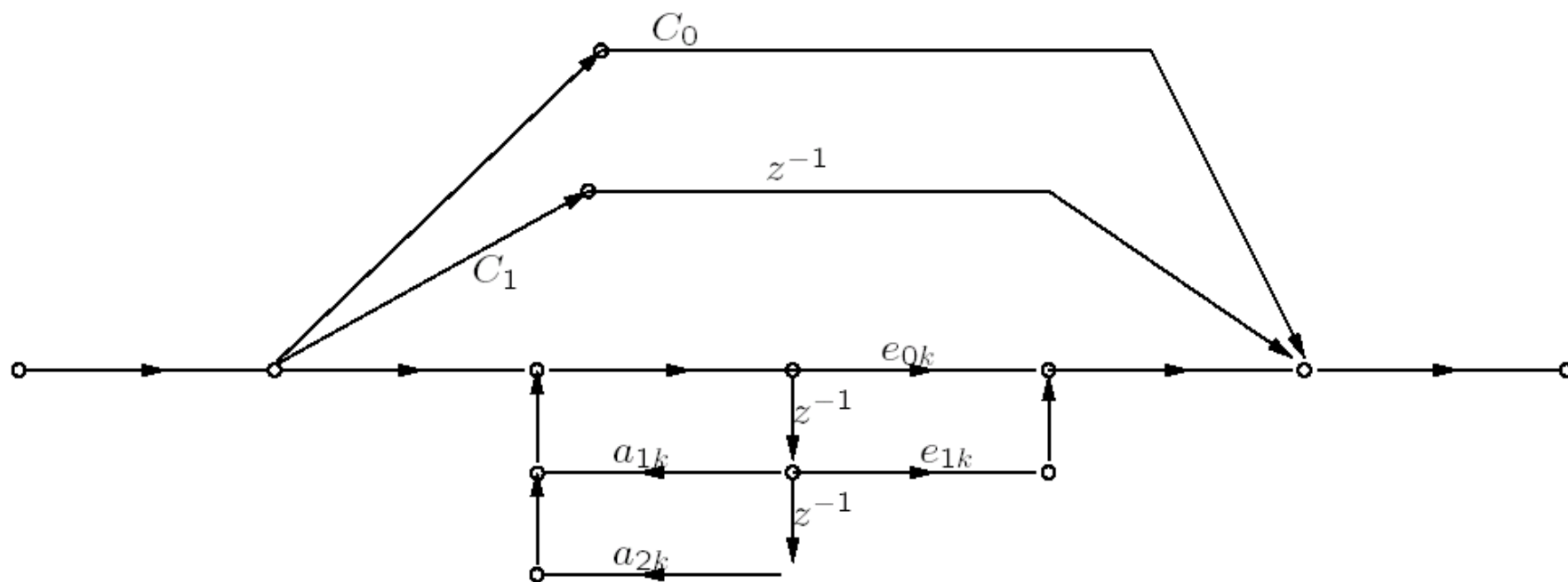
$$w_k[n] = a_{1k}w_k[n-1] + a_{2k}w_k[n-2] + y_{k-1}[n], \quad k = 1, 2, \dots, N_s,$$

$$y_k[n] = b_{0k}w_k[n] + b_{1k}w_k[n-1] + b_{2k}w_k[n-2], \quad k = 1, 2, \dots, N_s,$$

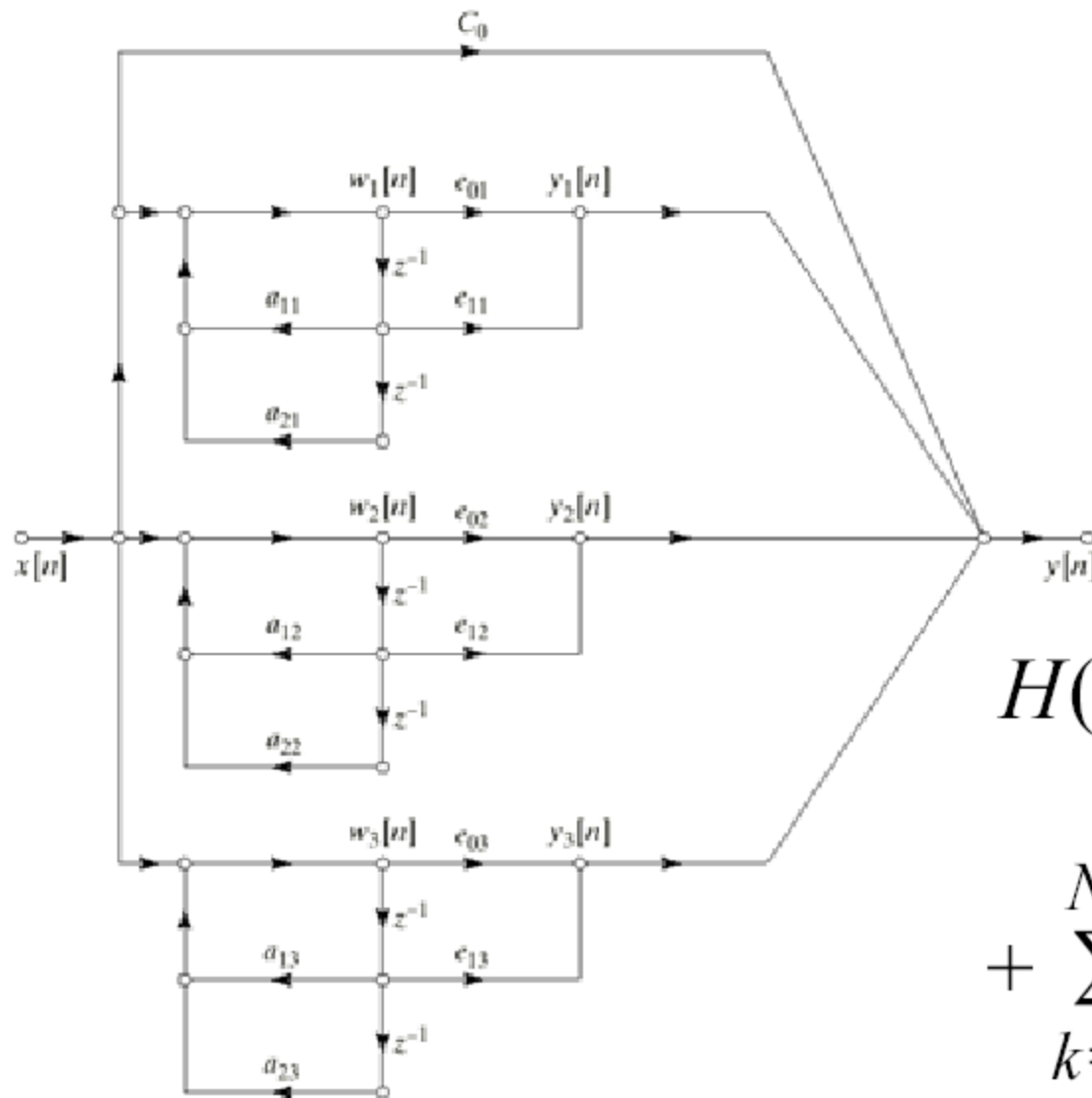
$$y[n] = y_{N_s}[n].$$

Parallel Form

$$\begin{aligned}
 H(z) &= \sum_{k=1}^{N_p} C_k z^{-k} + \sum_{k=1}^N \frac{A_k}{1 - d_k z^{-1}} \\
 &= \sum_{k=1}^{N_p} C_k z^{-k} + \sum_{k=1}^{N_s} \frac{e_{0k} + e_{1k} z^{-1}}{1 - a_{1k} z^{-1} - a_{2k} z^{-2}}
 \end{aligned}$$



Parallel Form



$$H(z) = \sum_{k=0}^{N_p} C_k z^{-k} + \sum_{k=1}^{N_s} \frac{e_{0k} + e_{1k} z^{-1}}{1 - a_{1k} z^{-1} - a_{2k} z^{-2}}$$

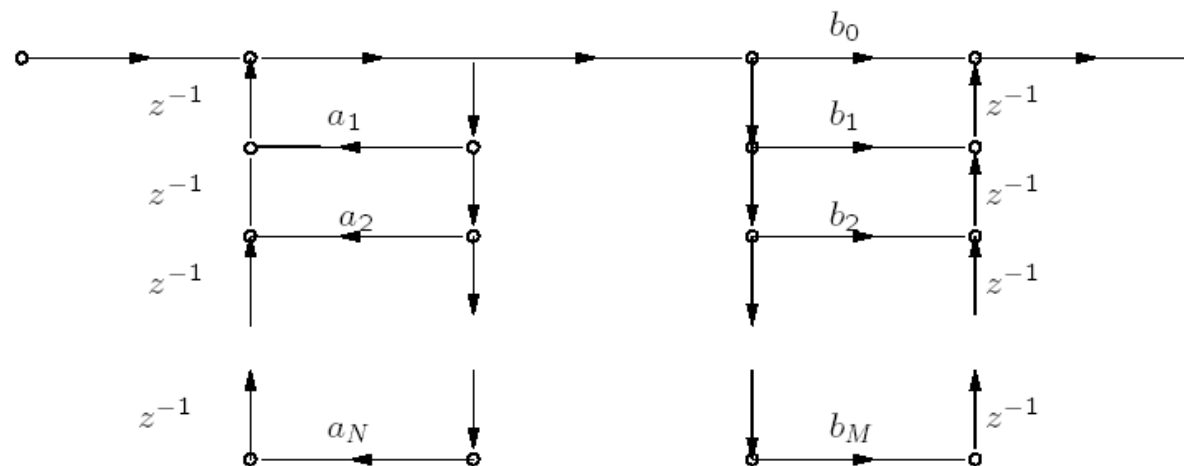
Transposed Forms

Transposition:

- Reverse all branches
- Swap input and output

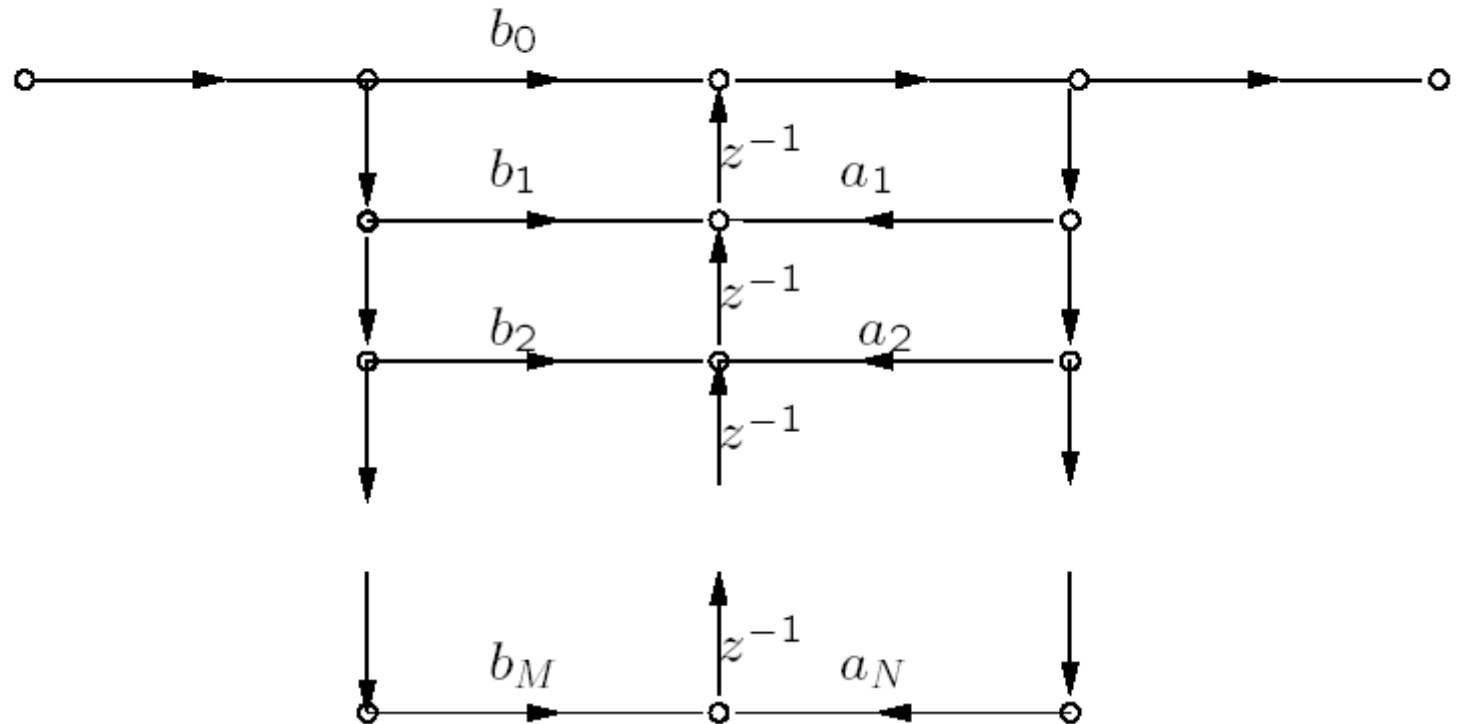
⇒ the same transmittance

Ex: Transposed DFI:



Transposed Forms

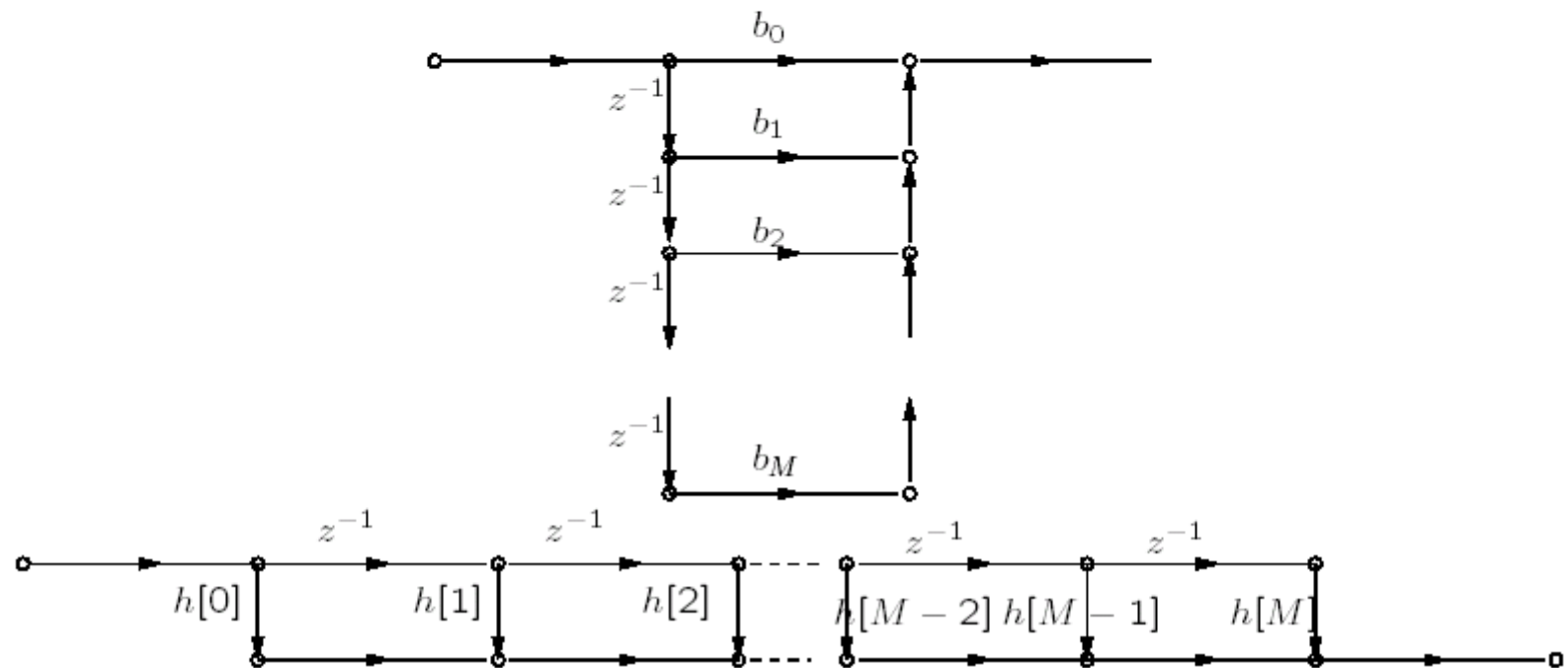
Ex: Transposed DFII:



Structures for FIR Filters

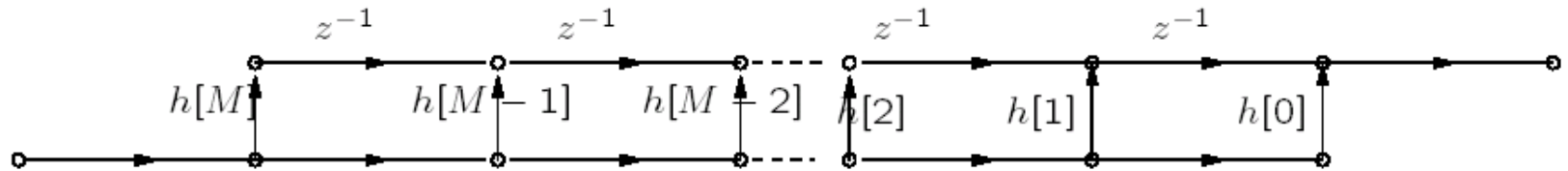
$$H(z) = \sum_{k=0}^M b_k z^{-k}$$

- only zeros, and poles at $z = 0$
- DFI=DFII=*transversal filter structure, tapped delay line*



Structures for FIR Filters

- transposed DF I:



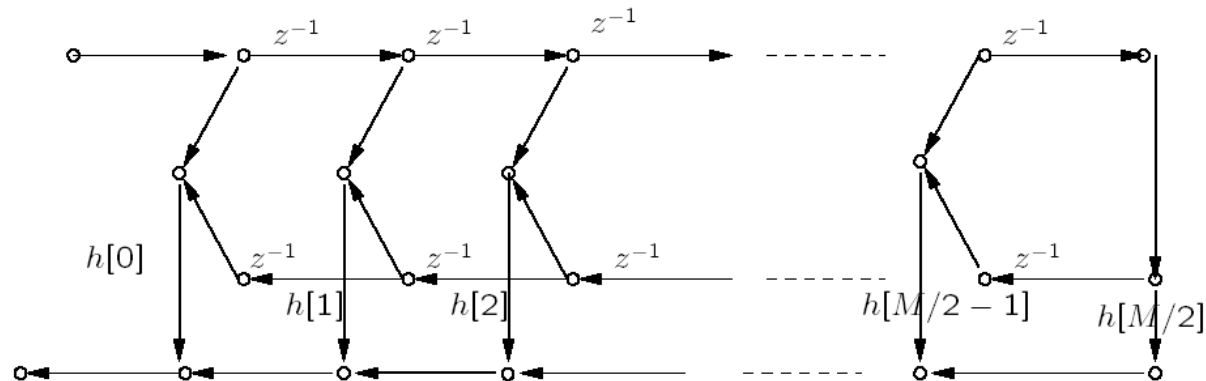
- cascade form possible with sections of the form

$$b_{0k} + b_{1k}z^{-1} + b_{2k}z^{-2}$$

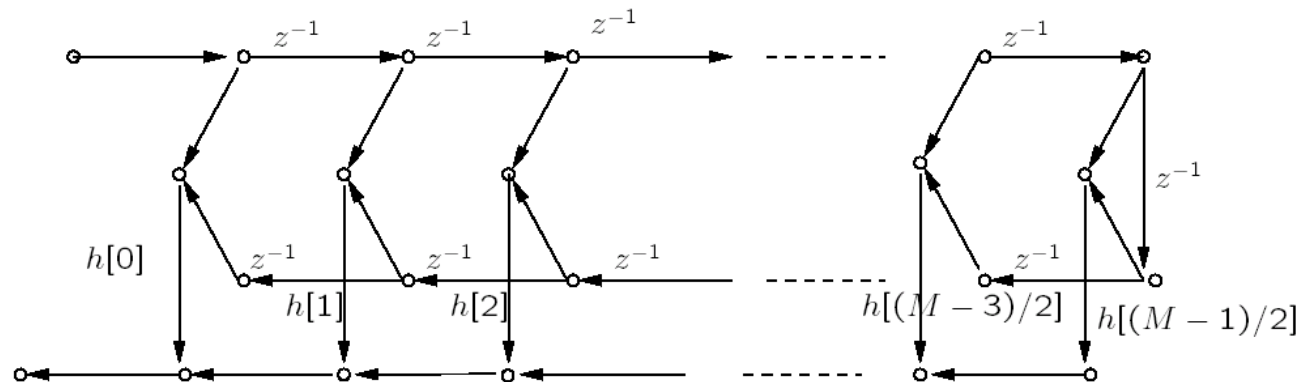
Structures for LP FIR Filters

- symmetry: $h[M - n] = \pm h[n] \Rightarrow$ half the number of coefficients

M even :



M odd :



Quantization Effects

- discrete-time filters, not digital filters.
- Most DSP systems are implemented using fixed-point arithmetic
- Floating-point arithmetic helps alleviate this problem, but consumes too much power and costs more
- Due to the very nature of DSP, where digital data are obtained through an A/D converter, floating-point precision is usually not required

Coefficient Quantization

- first design a discrete-time filter with double floating-point precision, such as the use of Matlab
- Truncate (or round) the filter coefficients to implement the fixed-point HW/SW

Finite Precision effects

- quantization of filter coefficients: $a \longrightarrow \hat{a} = Q(a)$
- truncation/rounding after multiplications and before additions
- input and output quantization

Effects:

- quantization of coefficients $\Rightarrow$ desired response $\neq$ actual response !
- round-off noise and overflow
- zero-input limit cycles (nonlinear effects)

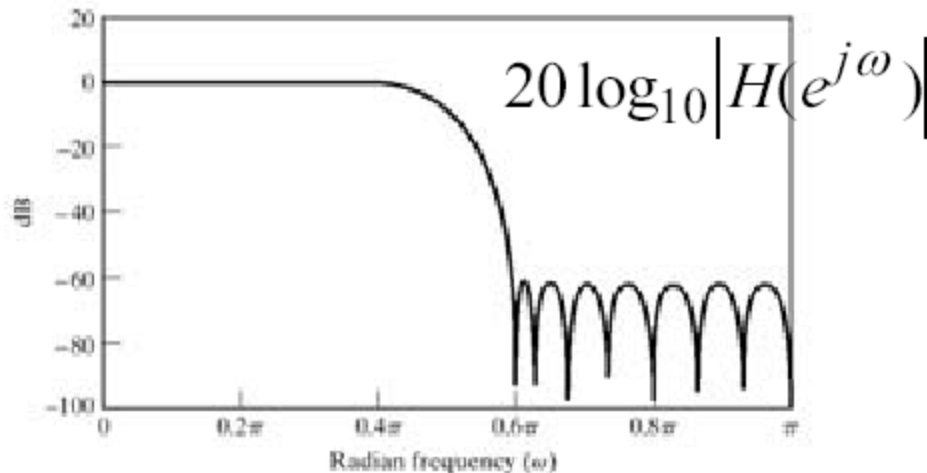
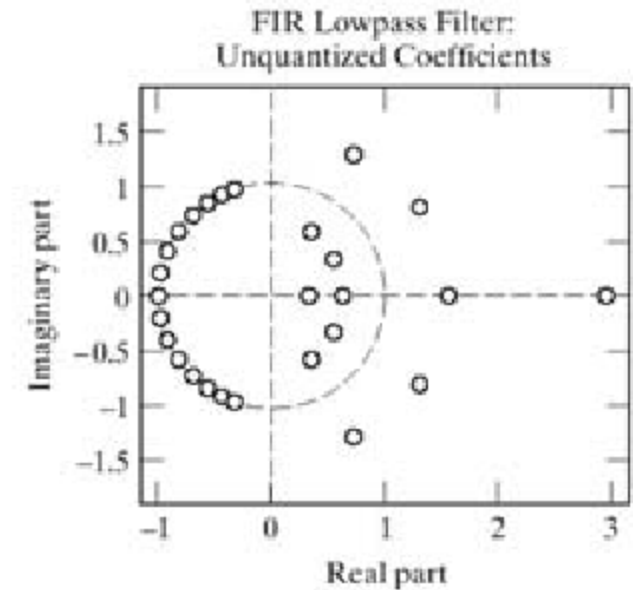
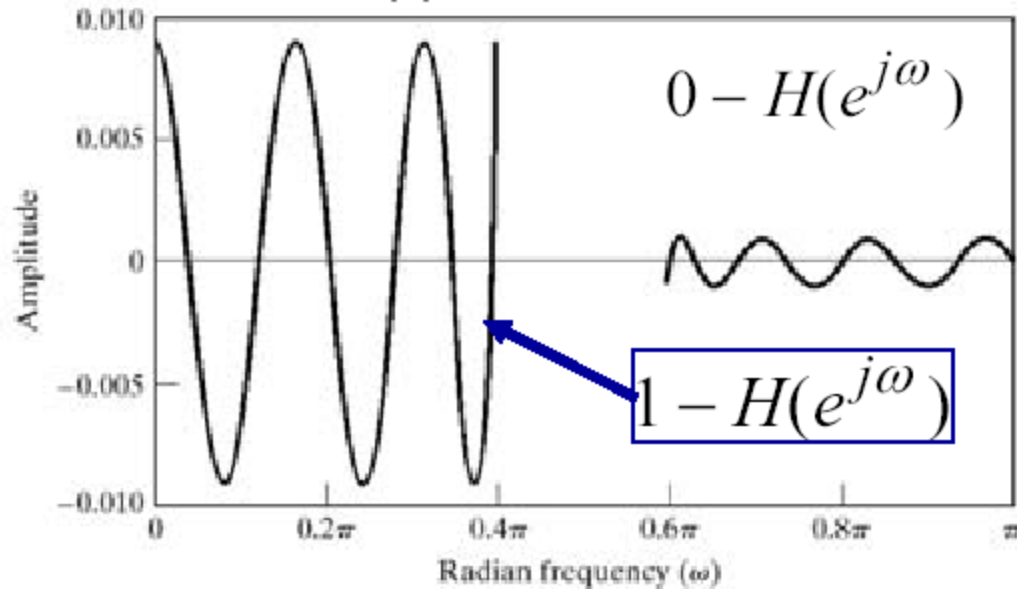
Quantization effects on the FIR systems

$$\hat{H}(z) = \sum_{n=0}^M [h(n) + \Delta h(n)]z^{-n} = H(z) + \Delta H(z)$$

$$\Delta H(e^{j\omega}) = \sum_{n=0}^M [\Delta h(n)]e^{-j\omega n} \leq \sum_{n=0}^M |(\Delta h(n))| |e^{-j\omega n}| \leq 2^{-(B+1)}(M+1)$$

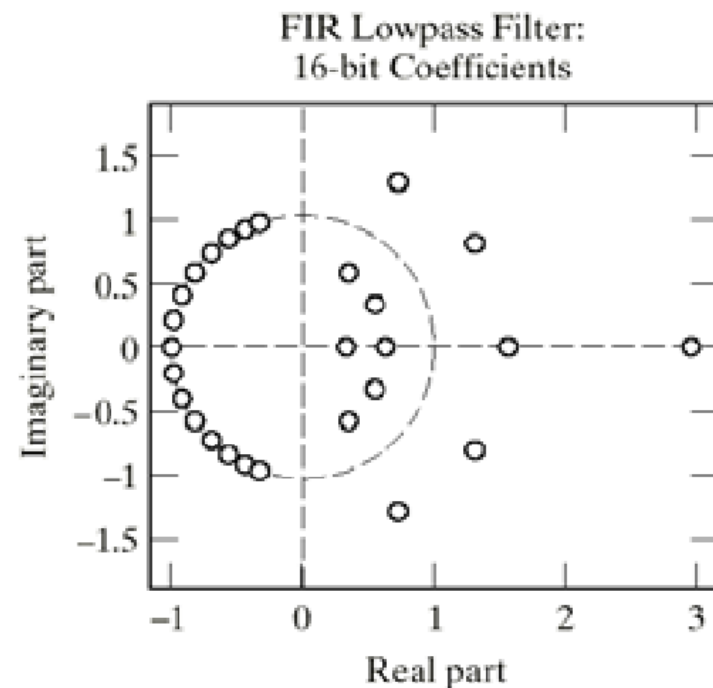
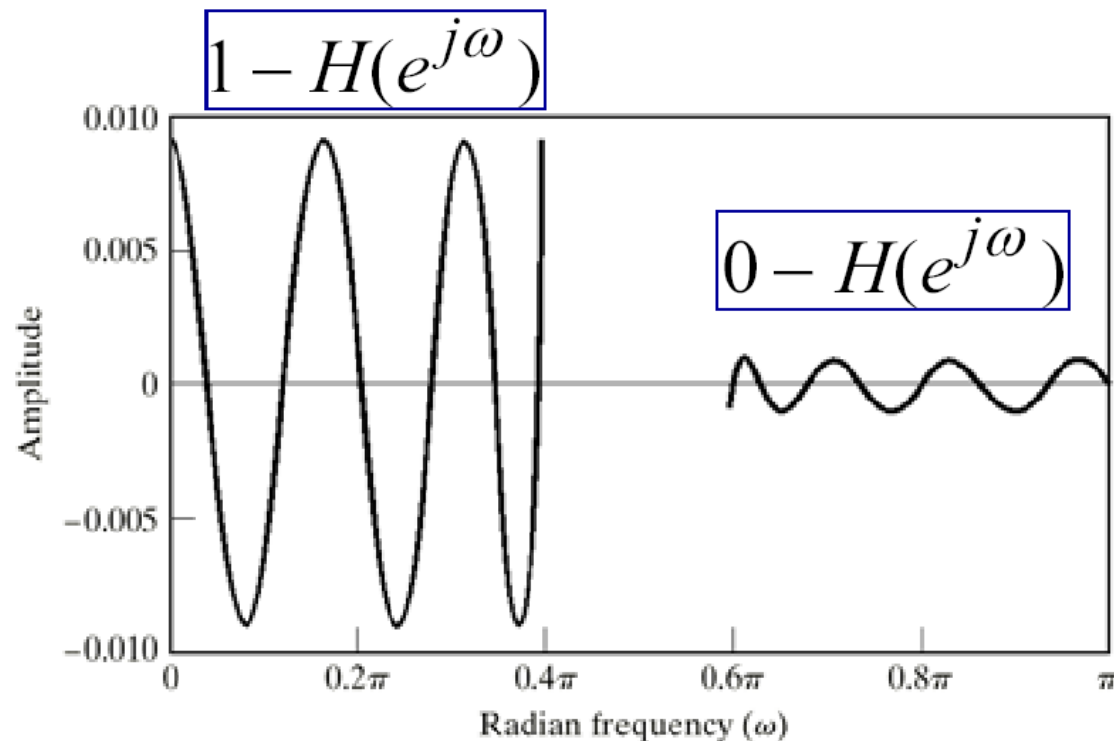
Unquantized FIR Filter Effect

Approximation error



Note that zeros are in conjugate reciprocal groups (1,2, or 4). This is a basic property of FIR linear phase filters.

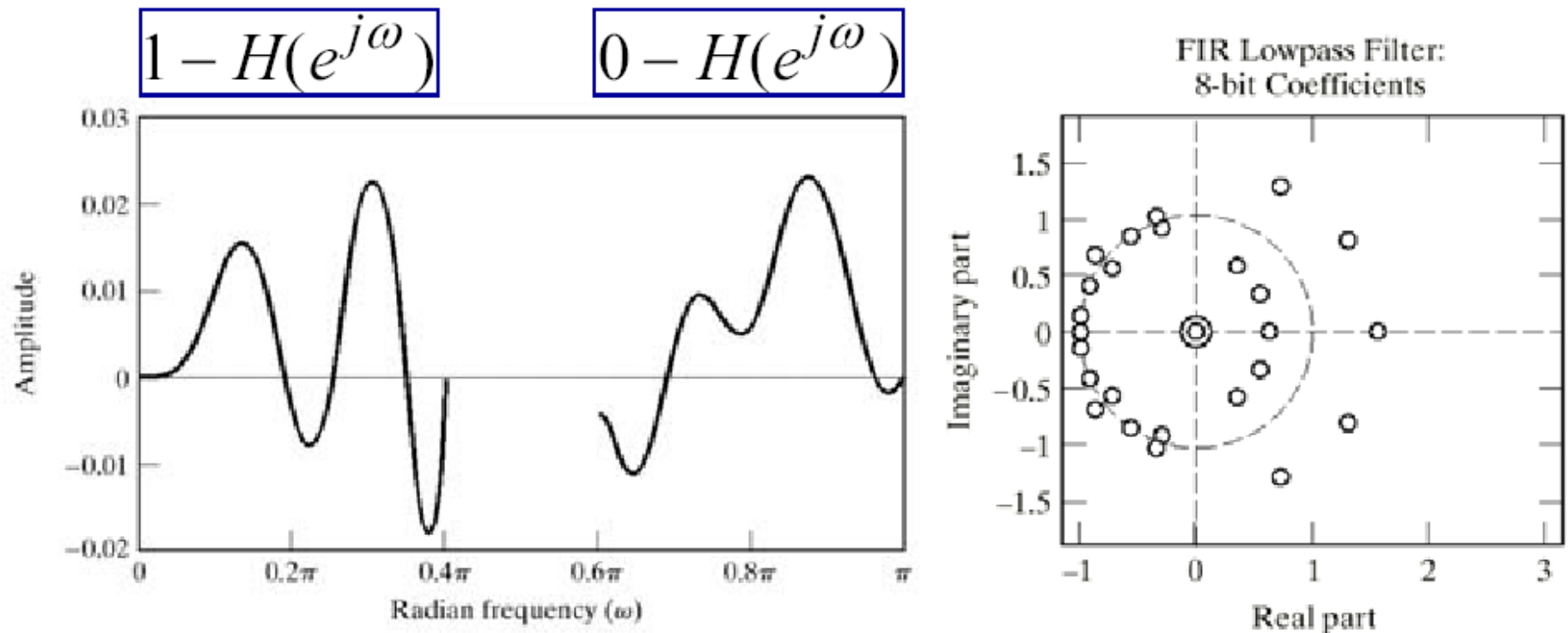
16-bit Quantization FIR Filter



We preserve the linear phase property since

$$\hat{h}[M - n] = \pm \hat{h}[n]$$

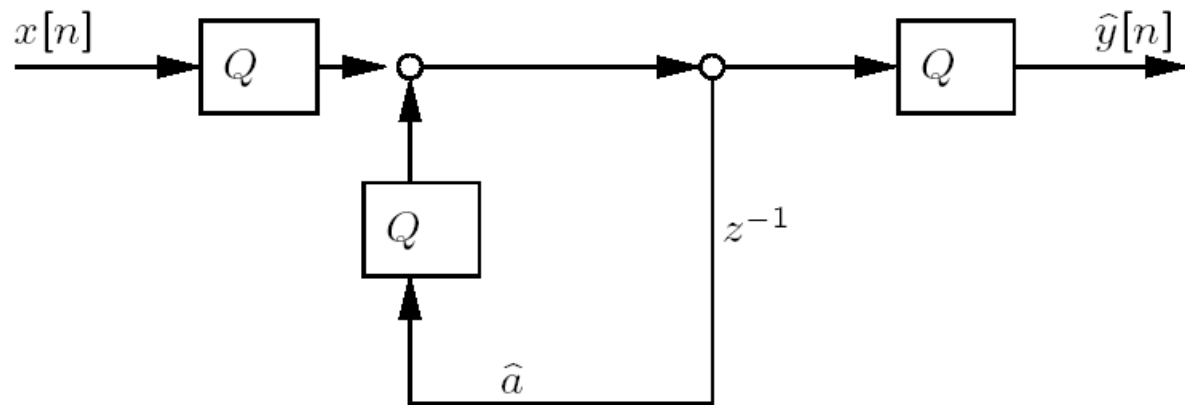
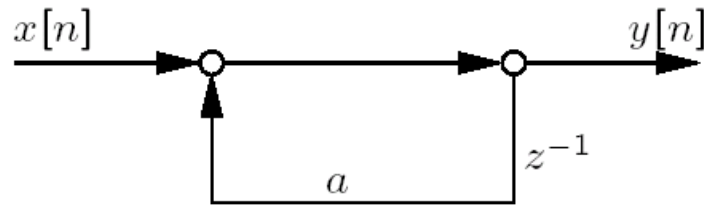
8-bit Quantization of FIR Filter



Zeros stay in reciprocal quads, but shift significantly due to the quantization.

Finite Precision effects– Example

$$H(z) = \frac{1}{1 - az^{-1}}$$



Coefficient Quantization

- Direct forms:
$$\hat{H}(z) = \frac{\sum_{k=0}^M \hat{b}_k z^{-k}}{1 - \sum_{k=1}^N \hat{a}_k z^{-k}}$$

$\Rightarrow$ all poles and zeros are influenced by quantization of each coeff. Quantization may cause high discrepancies between $H(z)$ and $\hat{H}(z)$, even more so if poles/zeros clustered.

- Cascade form:
$$\hat{H}(z) = \prod_{k=1}^{N_s} \frac{\hat{b}_{0k} + \hat{b}_{1k}z^{-1} + \hat{b}_{2k}z^{-2}}{1 - \hat{a}_{1k}z^{-1} - \hat{a}_{2k}z^{-2}}$$

$\Rightarrow$ Each pole-zero pair is affected only by quantization in its 2nd order section

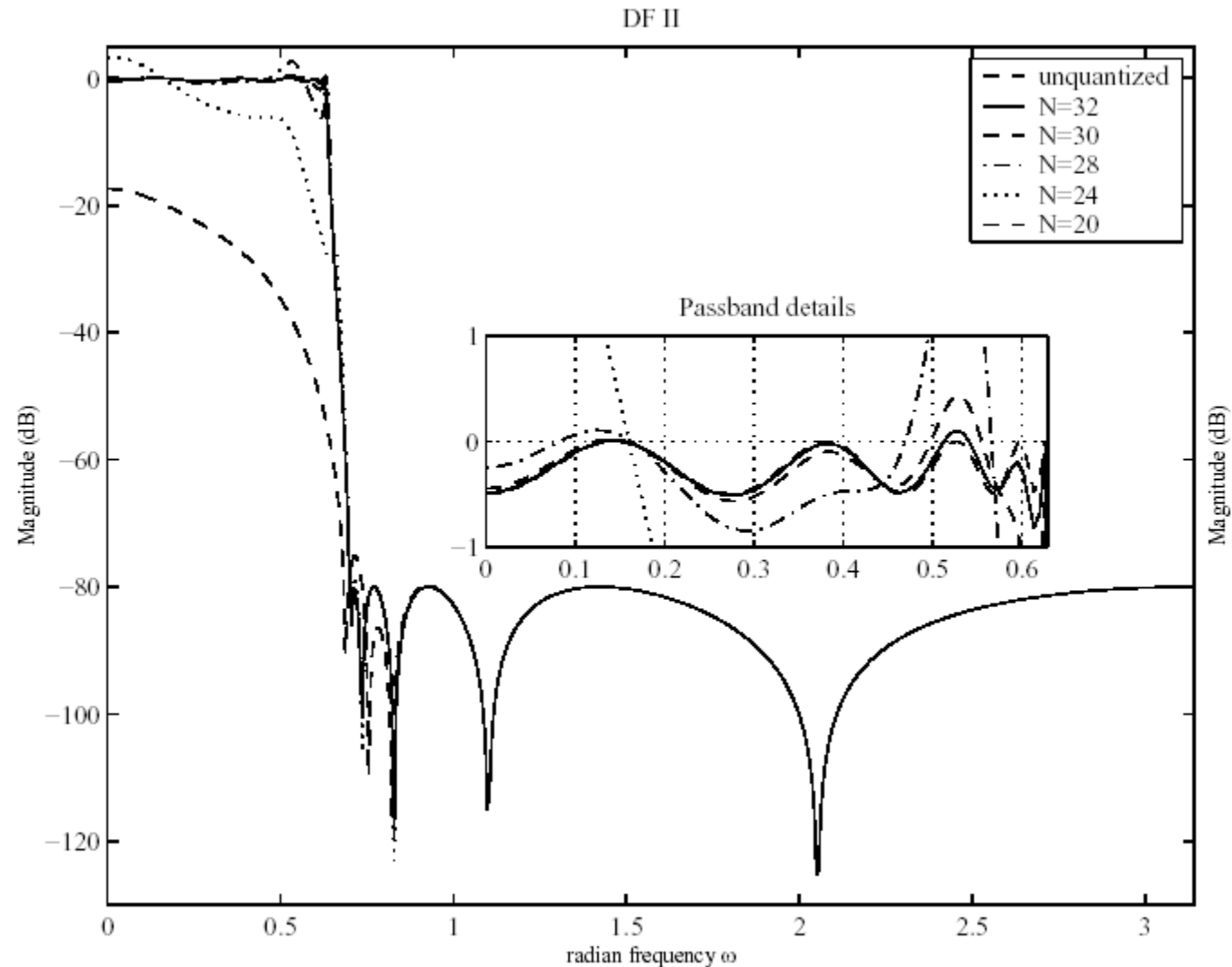
Coefficient Quantization(cont.)

- Parallel form:
$$\hat{H}(z) = \sum_{k=1}^{N_s} \frac{\hat{e}_{0k} + \hat{e}_{1k}z^{-1}}{1 - \hat{a}_{1k}z^{-1} - \hat{a}_{2k}z^{-2}}$$

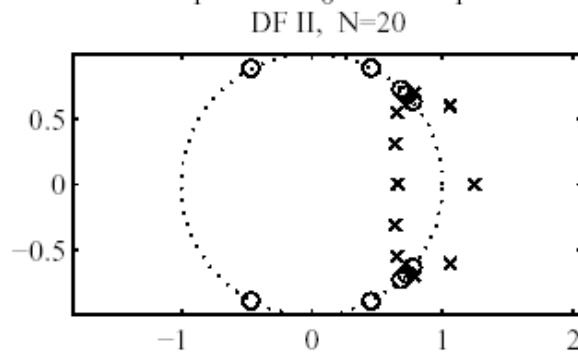
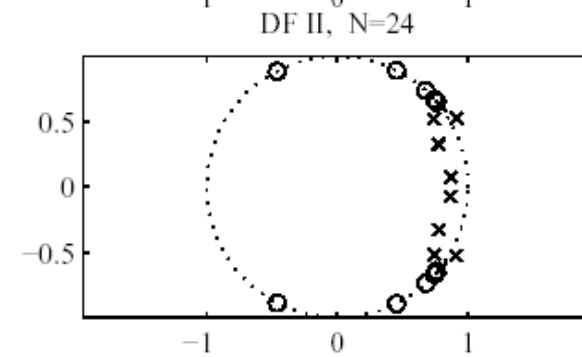
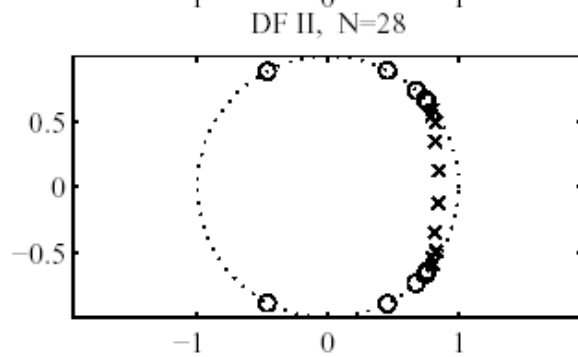
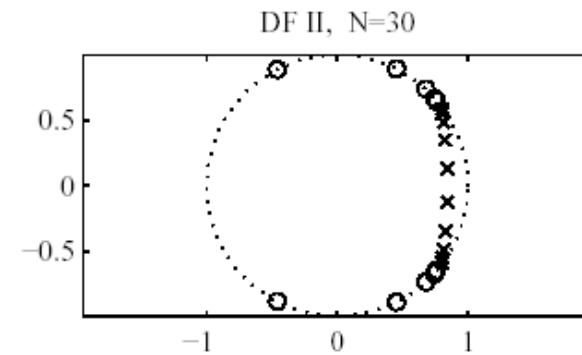
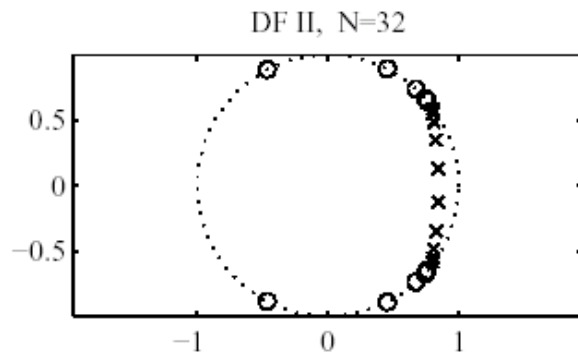
Each pole pair is affected only by quantization in its section, all zeros are affected by every quantization error.

DFII vs. 2nd order sections for IIR

IIR filter, order 10, $\omega_c = .2\pi$, design passband ripple .5 dB, attenuation 80 dB.



DFII vs. 2nd order sections for IIR



DFII vs. 2nd order sections for IIR

